

Actuarial & Life-Contingency Implementation Walkthrough

How the Go port computes life-contingent present values: the reconstructed mortality engine, the contingency combinations, the payment-on-death valuation, and how they weave into present value.

Scope. The Go actuarial package `internal/finance/actuarial/` (`contingency.go`, `table.go`) and its integration into present value (`calc.go`, `backward.go`, `variablerate.go`), plus the API wiring.

Provenance. Unlike the other engines, the actuarial computational core has **no DOS source to port** — the original ACTUARY unit is missing. The Go code is a *reconstruction* from the surviving integration points plus standard actuarial mathematics. See `legacy/code_docs/actuarial_dos_walkthrough.pdf` for the provenance analysis.

Audience. Engineers maintaining the Go port. Every claim is anchored to a `file:line` reference in `internal/`.

Contents

- 0 How to read this document**
- 1 The config & types**
- 2 The life-table mathematics**
- 3 Contingency combination & payment-on-death**
- 4 Integration into present value**
- 5 The per-payment installments column**
- 6 Appendix: truth table, data flow & file:line index**

0 How to read this document

The actuarial feature values annuities that continue “for life” and benefits paid “on death.” It is an overlay on the present-value engine: each cash flow is weighted by the probability the contingency holds at its payment date, and a payment-on-death (POD) term is added. Read the present-value Go walk-through first.

This is a reconstruction, not a port. The DOS ACTUARY unit (with `LifeProb`, `PODValue`, and the mortality table) is absent from the materials. The Go core is rebuilt to satisfy the *integration contract* recovered from the surviving call sites, against canonical actuarial mathematics. The contingency-type constants and integration semantics are anchored to Pascal; the table values, the intra-year interpolation, and the POD quadrature are reconstructions and cannot be diffed against the original.

Citation convention

References look like `contingency.go:142` and resolve under `internal/finance/actuarial/` unless another path is named (e.g. `presentvalue/calc.go:580`). Pascal lineage of an integration point is noted as `(PRESVALU.pas:NNN)`.

1 The config & types

ActuarialConfig `contingency.go:86–97` carries the inputs the missing DOS unit needed: up to two life tables and dates of birth, the valuation date Now (the “alive at now” anchor), the POD benefit amount, and a PODUnknown flag (when set, POD is the field to solve for and the input is ignored). The seven contingency constants `contingency.go:13–21` mirror the surviving Pascal (PETYPES.PAS:169); RequiresSecondLife `contingency.go:28–35` flags the four that reference person 2 (Only1/Only2/Either/Both). Parsers ContingencyLabel/ContingencyFromCode `contingency.go:38–79` map the N/L/D/1/2/E/B codes.

2 The life-table mathematics

A `LifeTable` [table.go:30–34](#) holds Qx (death probability within a year) and Lx (survivors from a 100,000 radix). The core operations:

$$l(\text{age}) = Lx[\lfloor \text{age} \rfloor] \cdot (1 - \text{frac}) + Lx[\lfloor \text{age} \rfloor + 1] \cdot \text{frac} \quad (\text{linear interpolation, UDD}) \quad \text{— } \text{table.go:44}$$

$$\text{SurvivalProb}(\text{age}) = l(\text{age}) / Lx[0] \quad \cdot \quad \text{ConditionalSurvival}(x, x+t) = {}_t p_x = l(x+t) / l(x)$$

`SurvivalProb` [table.go:44–65](#) uses **linear Lx interpolation** between integer ages (a uniform-distribution-of-deaths assumption); `ConditionalSurvival` [table.go:70–80](#) is the survival probability conditional on being alive at the current age. `LifeExpectancy` [table.go:85](#) sums forward conditional survival to the max age. Loaders `NewLifeTableFromQx/FromLx` [table.go:101 / 116](#) build one array from the other ($Lx[i+1] = Lx[i] \cdot (1 - Qx[i])$); the API uses `ParseJSON` [table.go:208](#). The table is **data-driven** precisely because the original DOS values are unknown.

3 Contingency combination & payment-on-death

Per-person survival is conditional on being alive at Now: `survivalProb1/2(date)` [contingency.go:114–132](#) = `ConditionalSurvival(age@Now, age@date)`, with ages via a 360-day `yearsDif`. Critically, `survivalProb2` returns 1.0 when no second table is set — a silent “immortal” default that the two-life guard (§4) exists to catch.

`LifeProb(date, contingency)` [contingency.go:142–166](#) combines one or two independent lives (s_1, s_2):

Living: s_1 · Dead: $1-s_1$ · Only1: $s_1(1-s_2)$ · Only2: $(1-s_1)s_2$
 Either (last-survivor): $1-(1-s_1)(1-s_2)$ · Both (joint-life): $s_1 \cdot s_2$

Living + Dead = 1 — a complementarity the engine relies on (see §5). The payment-on-death value `PODValueFunc(asOf, discountForYears)` [contingency.go:203–238](#) is a discrete death-year summation with a mid-year discount:

$$\text{PODValue} = \sum_k \text{POD} \cdot [{}_k p - {}_{k+1} p] \cdot \text{discount}(\text{offset} + k + \tfrac{1}{2})$$

where $[{}_k p - {}_{k+1} p]$ is the probability of death in year k and the discount is a **callback** — so the fixed-rate path passes $e^{-\text{rate} \cdot \text{years}}$ (`PODValue` [contingency.go:174](#)) while the variable-rate path discounts each death year through its schedule. The result is rounded with `interest.Round2`.

4 Integration into present value

The integration is a clean symmetry that mirrors the recovered DOS contract: the survival weight and the POD term are *added* on the forward pass and *removed* on the backward pass.

Forward weighting

A lump value is multiplied by its survival probability `presentvalue/calc.go:489–495` (PRESVALU.pas:212). A periodic stream routes through `periodicWithActuarial` `presentvalue/calc.go:580`, which **forces exact period-by-period summation** and weights each payment by `LifeProb(t)` at its own date (PRESVALU.pas:290, 297) — the closed-form series cannot carry a per-period weight. The POD value is added to the total `presentvalue/calc.go:551` (PRESVALU.pas:689).

The two-life guard

Because `survivalProb2` silently treats a missing person 2 as immortal, any row using a `RequiresSecondLife` contingency without a usable second table + DOB is rejected by name up front — `checkSecondLifeProvided` `presentvalue/calc.go:380–402`, called from both `Calculate` and the API.

Backward symmetry & the date-solve prohibition

Backward solves invert the *same weighted* factor: `computeKnownRowSum` adds the POD value into the known subtotal so it is peeled from the target `presentvalue/backward.go:631` (PRESVALU.pas:848), and amount solves divide the survival probability back out (erroring when `prob ≤ teeny` — payment beyond the table horizon). A life-contingent **date** solve is rejected outright `presentvalue/backward.go:725–734` (`no_time_with_life`, PRESVALU.pas:894) because the survival probability itself depends on the unknown date. The unknown-POD solve `presentvalue/calc.go:412` exploits linearity in the benefit amount: it probes `POD = 0` and a large value, then divides — the same trick the variable-rate `solveVariableRatePOD` uses `presentvalue/variableRate.go:495`.

5 The per-payment installments column

A recent change re-aligned the Go output to DOS's per-payment probability column. DOS's table (`pvltable.pas` `PrintNextPayment`) prints one row per payment date, each with its own `LifeProb(t)` and an "if paid" value; the Go engine previously reported only a stream *average*. Now `PeriodicPayment` carries an `Installments []PeriodicInstallment` slice (`presentvalue/types.go:69–89`) — per payment: the date, `IfPaid` (discounted but not survival-weighted — DOS `ifpd`), `Prob` (`LifeProb(t)`), and `Value = IfPaid · Prob` (DOS `v`).

It is built inside `periodicWithActuarial` (`presentvalue/calc.go:580–670`) (dollar-scaled by the payment amount) and the variable-rate `vrPeriodicValue` (`presentvalue/variableRate.go:159`), populated only for life-contingent rows, and surfaced via the API as `PVInstallmentResp` (`api/handlers.go:468`). The row's `Prob` remains the stream average for back-compat; the installments are the DOS-faithful representation.

Why the loop never early-breaks. DOS's exact summation breaks when a term falls below teeny, with a special guard for death-conditioned contingencies. The Go loop instead never early-breaks (`presentvalue/calc.go:648–657`), because `Dead / Only1 / Only2 / Either` probabilities are *non-monotone* — near zero for a young insured, growing over time — so an early break would zero out their value and violate the `Living + Dead = non-contingent complementarity`. A regression test (`presentvalue/actuarial_installments_test.go:23`) locks in that each installment's `Prob` equals `LifeProb(date)`, that `Value = IfPaid · Prob`, that the installment values reconcile to the row `Val`, and that the row average is the mean of the per-payment probabilities.

Backward POD peel / date-solve reject / unknown-POD	<code>presentvalue/backward.go:631 / 725 · calc.go:412</code>
<code>periodicWithActuarial / installments type / test</code>	<code>presentvalue/calc.go:580 · types.go:69 · actuarial_installments_test.go:23</code>
<code>buildActuarialConfig / PVInstallmentResp</code>	<code>api/handlers.go:1547 / 468</code>

Per%Sense Go port. Citations resolve under `internal/`. For the provenance analysis (what is reconstructed vs recovered) and the DOS integration contract, see `legacy/code_docs/actuarial_dos_walkthrough.pdf`. This feature overlays the present-value engine — see the present-value Go walk-through.